

UDP Client Experiment

UDP Client Experiment

1. UDP Client Overview
2. Core Concepts
 - 2.1 UDP vs TCP
 - 2.2 UDP Send Three-Step Process
 - 2.3 Receiving Characteristics
3. Hardware Dependencies
4. Project Architecture and Flow
5. Source Code Details
 - 5.1 Configuration and Global Variables
 - 5.2 `setup()` — Bind Local Port
 - 5.3 `loop()` — Send and Receive
6. Operation Flow
 - 6.1 Build and Flash
 - 6.2 Operation Steps
 - 6.3 Verification Methods
7. Common Problems

1. UDP Client Overview

UDP (User Datagram Protocol) is a **connectionless, unreliable but efficient** transport layer protocol. Unlike TCP, UDP does not establish connections, does not guarantee delivery, and does not guarantee order, but has **low overhead and low latency**, making it very suitable for scenarios with high real-time requirements that allow occasional packet loss.

In IoT applications, UDP is widely used for **high-frequency sensor data reporting** (e.g., temperature and humidity collected 100 times per second, where losing a few frames is acceptable), **LAN device discovery** (broadcast/multicast probing for online devices on the same subnet), **DNS domain resolution** (small query volume, retry on timeout), and **real-time audio/video streaming** (latency takes priority over picture quality integrity).

In the Arduino ESP32 development environment, the system provides the packaged `WiFiUDP` library, which is still based on the lwIP protocol stack's `sendto/recvfrom` interface underneath, greatly simplifying the UDP programming process. This experiment configures ESP32-S3 as a UDP client. The device reports data to a specified UDP server at a 1-second cycle while listening and parsing server response packets in real-time.

The experiment uses the unified `WiFiUDP` standard three-step transmission mechanism: use `beginPacket()` to specify the communication target address and port, use `write()` to fill the data payload to be transmitted, and finally call `endPacket()` to complete the UDP datagram transmission, fully implementing lightweight, high real-time IoT UDP data interaction.

Typical Application Scenarios:

Application Type	Example
Real-time Streaming	Audio/video transmission (losing a few frames does not affect experience)
Domain Query	DNS requests
Clock Synchronization	NTP time synchronization
Sensor Broadcasting	Multiple devices simultaneously receiving sensor data

2. Core Concepts

2.1 UDP vs TCP

Feature	UDP	TCP
Connection	Connectionless, send packets directly	Requires three-way handshake
Reliability	No delivery guarantee, no retransmission	ACK confirmation + automatic retransmission
Message Boundary	Preserved (one send = one datagram)	Streaming (data coalescing, needs packet splitting)
Applicable Scenarios	Video streaming, DNS, IoT sensor data	Web pages, file transfer, MQTT
Transmission Efficiency	High (header only 8 bytes)	Lower (header 20 bytes)

2.2 UDP Send Three-Step Process

`beginPacket(ip, port)` specify target → `write(buf, len)` fill payload → `endPacket()` encapsulate UDP header + IP header and send. Underneath, it calls lwip's `sendto()`.

2.3 Receiving Characteristics

UDP naturally preserves message boundaries — each `parsePacket() + read()` exactly completes reading one complete datagram, without the packet coalescing problem that TCP has.

3. Hardware Dependencies

Pure software experiment. You must first connect to the router via WIFI-STA, and the client and server need to be on the same local area network.

4. Project Architecture and Flow

```
setup()
├─ connectWiFi()           // Connect to router (same as WIFI-STA experiment)
└─ udp.begin(LOCAL_PORT)  // Bind local port

loop()
├─ Every second: beginPacket → write → endPacket // Send datagram
├─ parsePacket() >= 0 → read()                 // Non-blocking read response
└─ delay(10)
```

5. Source Code Details

Complete source code: [Source code → Arduino → 3.Network Communication → UDP_Client → UDP_Client.ino](#)

5.1 Configuration and Global Variables

```
#include <WiFiUDP.h>
WiFiUDP udp;
const char* SERVER_IP = "192.168.11.183";
uint16_t SERVER_PORT = 8080;
uint16_t LOCAL_PORT = 8080;
```

5.2 `setup()` — Bind Local Port

```
void setup() {
  Serial.begin(115200);
  connectWiFi();           // Same as WIFI-STA experiment
  udp.begin(LOCAL_PORT);   // Underneath: socket() → bind(8080)
}
```

5.3 `loop()` — Send and Receive

Send a datagram every second while non-blockingly checking for responses.

```
void loop() {
  static unsigned long lastSend = 0;
  if (millis() - lastSend > 1000) {
    lastSend = millis();
    udp.beginPacket(SERVER_IP, SERVER_PORT); // Specify target
    udp.write("ESP32 UDP Client Data");      // Fill payload
    udp.endPacket();                          // Send
  }

  int packetSize = udp.parsePacket(); // Non-blocking check
  if (packetSize > 0) {
    char buf[128];
```

```
int len = udp.read(buf, sizeof(buf) - 1);  
buf[len] = '\0';  
Serial.printf("UDP Receive: %s\n", buf);  
}  
delay(10);  
}
```

`parsePacket()` returns the number of bytes in the datagram, 0 means no data. Unlike TCP, each `parsePacket + read` only reads one complete datagram.

6. Operation Flow

6.1 Build and Flash

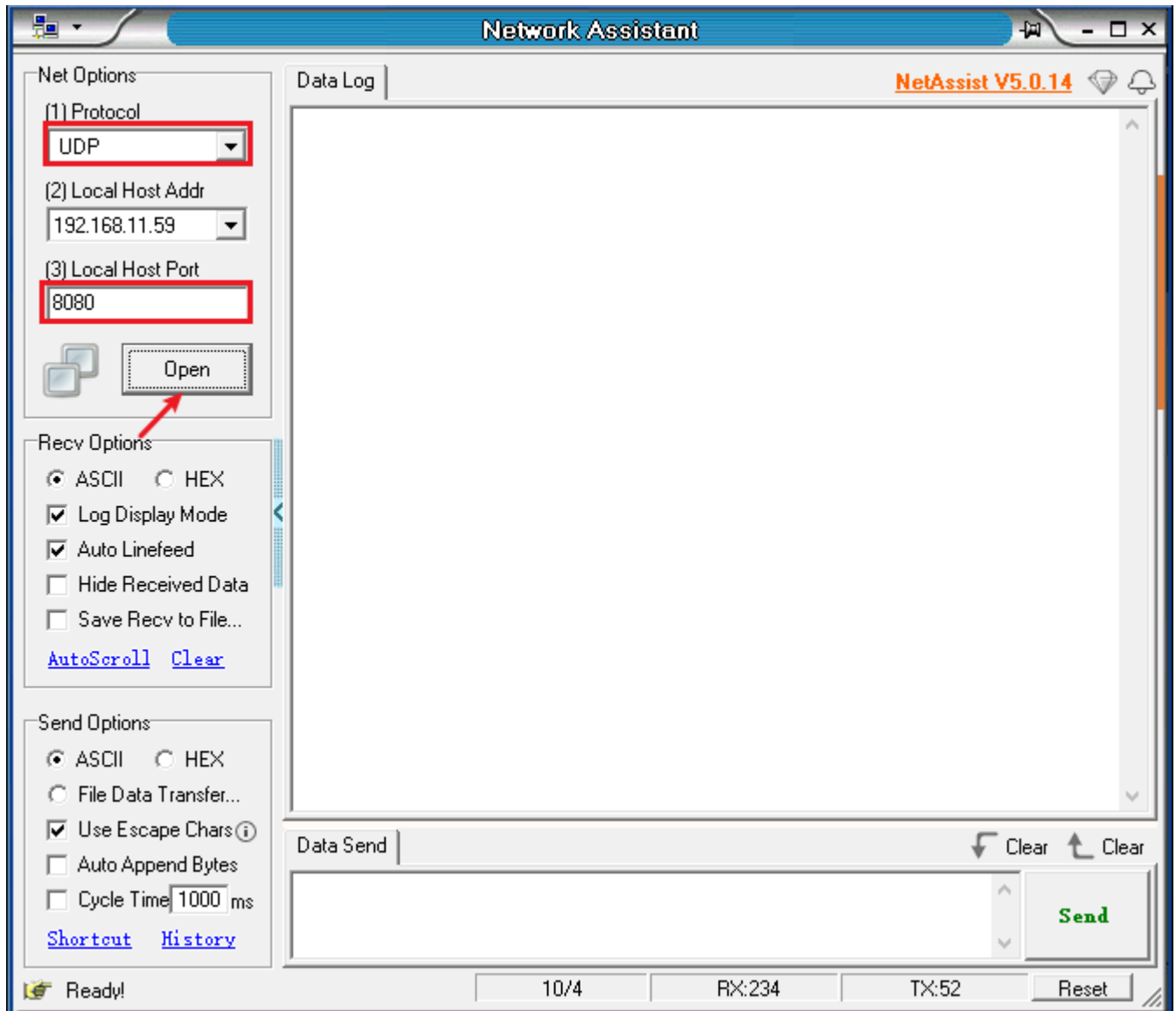
Arduino flash process: See `Arduino → 1. Before Development → 2. Build and Flash`.

1. Modify the server IP to the actual address
2. Build and flash

6.2 Operation Steps

Step 1: Open the network debugging assistant  **, configure UDP server**

Open the network debugging assistant on the PC and configure it with the following parameters:



Parameter	Value
Protocol Type	UDP
Local IP Address	PC's LAN IP (e.g., 192.168.11.183)
Local Port	8080

Click the "Connect" button, the network debugging assistant starts listening on UDP port 8080 and waits for client data.

```
[2026-08-06 19:05:23.936]# RECV ASCII/21 from 192.168.11.181 :8080 <<<
ESP32 UDP Client Data

[2026-08-06 19:05:24.939]# RECV ASCII/21 from 192.168.11.181 :8080 <<<
ESP32 UDP Client Data

[2026-08-06 19:05:25.940]# RECV ASCII/21 from 192.168.11.181 :8080 <<<
ESP32 UDP Client Data

[2026-08-06 19:05:26.940]# RECV ASCII/21 from 192.168.11.181 :8080 <<<
ESP32 UDP Client Data

[2026-08-06 19:05:27.938]# RECV ASCII/21 from 192.168.11.181 :8080 <<<
ESP32 UDP Client Data

[2026-08-06 19:05:28.947]# RECV ASCII/21 from 192.168.11.181 :8080 <<<
ESP32 UDP Client Data

[2026-08-06 19:05:29.946]# RECV ASCII/21 from 192.168.11.181 :8080 <<<
ESP32 UDP Client Data

[2026-08-06 19:05:30.953]# RECV ASCII/21 from 192.168.11.181 :8080 <<<
ESP32 UDP Client Data

[2026-08-06 19:05:31.942]# RECV ASCII/21 from 192.168.11.181 :8080 <<<
ESP32 UDP Client Data

[2026-08-06 19:05:32.949]# RECV ASCII/21 from 192.168.11.181 :8080 <<<
ESP32 UDP Client Data
```

Step 2: Modify the server IP in the code, flash the program

Open Arduino IDE, change `SERVER_IP` in the code to the PC's actual LAN IP address, then compile and flash to the ESP32-S3 development board.

Step 3: Open the serial monitor and observe the startup process

After flashing, open the serial monitor in Arduino IDE (baud rate 115200). You should see ESP32 connect to WiFi and start the UDP client:

```
19:06:39.068 -> Connecting to WiFi: Yahboom2 (Attempt 1)
19:06:40.125 -> ...
19:06:41.094 -> WiFi Connected successfully!
19:06:41.140 -> Device IP: 192.168.11.181
19:06:41.140 -> Target Server: 192.168.11.59:8080
19:06:41.140 -> UDP Client Startup successful!
19:06:41.140 -> UDP Send: ESP32 UDP Client Data
19:06:42.098 -> UDP Send: ESP32 UDP Client Data
...
```

At this point, the network debugging assistant should continuously receive `ESP32 UDP Client Data` sent by ESP32, indicating that UDP one-way communication is normal.

```

19:06:39.068 -> Connecting to WiFi: Yahboom2 (Attempt 1)
19:06:40.125 -> ...
19:06:41.094 -> WiFi Connected successfully!
19:06:41.140 -> Device IP: 192.168.11.181
19:06:41.140 -> Target Server: 192.168.11.59:8080
19:06:41.140 -> UDP Client Startup successful!
19:06:41.140 -> UDP Send: ESP32 UDP Client Data
19:06:42.098 -> UDP Send: ESP32 UDP Client Data
19:06:43.099 -> UDP Send: ESP32 UDP Client Data

```

Step 4: Reply data from the network debugging assistant to verify two-way communication

Enter `Hello YAHBOOM` in the send area of the network debugging assistant and click send. The serial monitor will print the received reply in real-time:

```

19:08:08.197 -> UDP Send: ESP32 UDP Client Data
19:08:09.216 -> UDP Send: ESP32 UDP Client Data
19:08:09.388 -> UDP Receive: Hello YAHBOOM
19:08:10.080 -> UDP Receive: Hello YAHBOOM
19:08:10.126 -> UDP Receive: Hello YAHBOOM
19:08:10.218 -> UDP Send: ESP32 UDP Client Data
19:08:11.188 -> UDP Send: ESP32 UDP Client Data
19:08:11.375 -> UDP Receive: Hello YAHBOOM
19:08:12.207 -> UDP Send: ESP32 UDP Client Data
...

```

```

19:08:07.212 -> UDP Send: ESP32 UDP Client Data
19:08:08.197 -> UDP Send: ESP32 UDP Client Data
19:08:09.216 -> UDP Send: ESP32 UDP Client Data
19:08:09.388 -> UDP Receive: Hello YAHBOOM
19:08:10.080 -> UDP Receive: Hello YAHBOOM
19:08:10.126 -> UDP Receive: Hello YAHBOOM
19:08:10.218 -> UDP Send: ESP32 UDP Client Data
19:08:11.188 -> UDP Send: ESP32 UDP Client Data
19:08:11.375 -> UDP Receive: Hello YAHBOOM
19:08:12.207 -> UDP Send: ESP32 UDP Client Data
19:08:13.222 -> UDP Send: ESP32 UDP Client Data
19:08:14.191 -> UDP Send: ESP32 UDP Client Data

```

For every `Hello YAHBOOM` sent, the serial monitor prints one `UDP Receive: Hello YAHBOOM`, proving that both ESP32 sending and receiving are working normally.

6.3 Verification Methods

Operation	Expected Result
Server online	Receive reply after sending
Server offline	Sending normal (UDP does not check if the other party exists), but no reply

7. Common Problems

Phenomenon	Cause	Solution
Sending normal but cannot receive reply	Server not running or wrong port	Confirm server IP/port, confirm server is started
<code>parsePacket</code> always returns 0	Local port inconsistent with server reply target	Check if the port bound by <code>begin()</code> is correct